

SINGLETON APLICADO EN IMPRESORAS

Patrones de Diseño: *Singleton y Multiton*

Gestión de conexiones y cola de impresión
con control de instancias

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Septiembre 2026

Índice

1. Diagrama UML	2
2. Explicación de las Clases	3
2.1. Entidades Base	3
2.1.1. Clase Archivo	3
2.1.2. Clase Impresora	3
2.2. Patrón Singleton - Conexion	5
2.3. Patrón Multiton - ColaImpresion	6
3. Código de Cliente	7
3.1. Clase Main	7
4. Pruebas y Ejecución	8
4.1. Prueba de Ejecución en Consola	8
5. Repositorio	9
5.1. Enlace GitHub	9
5.2. Estructura de Paquetes	9

Diagrama UML

Descripción General

El proyecto implementa los patrones de diseño **Singleton** para gestionar una única conexión activa con una impresora y **Multiton** para limitar las instancias de la cola de impresión a un número máximo permitido.

Diagrama de Clases

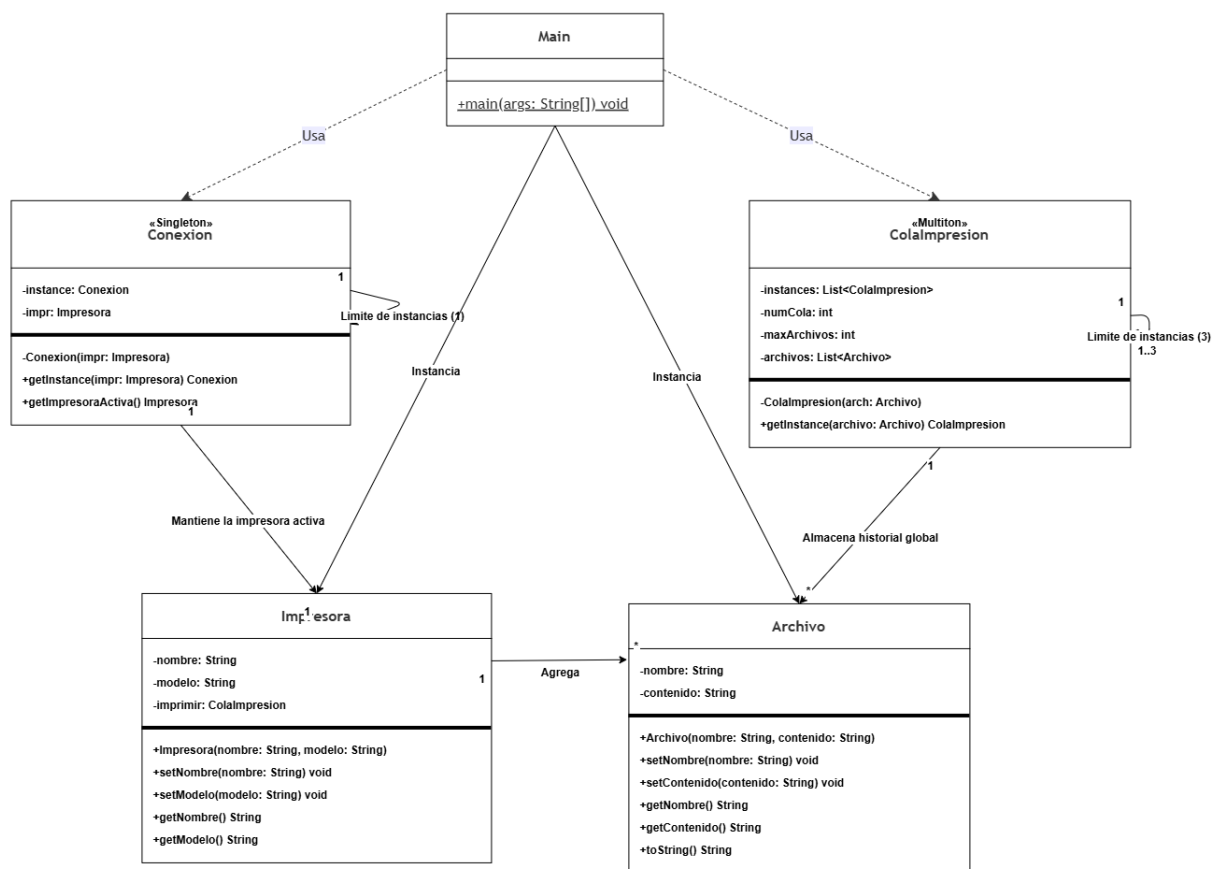


Figura 1: Diagrama de Clases del Sistema.

Explicación de las Clases

Entidades Base

2.1.1. Clase Archivo

Descripción

Representa los documentos que serán enviados a la cola de impresión. Contiene propiedades básicas para su identificación y contenido.

Código Fuente

```
1 package impresora.singleton;
2
3 public class Archivo {
4
5     private String nombre;
6     private String contenido;
7
8     public Archivo(String nombre, String contenido) {
9         this.nombre = nombre;
10        this.contenido = contenido;
11    }
12
13    public void setNombre(String nombre) { this.nombre = nombre;
14        }
15    public void setContenido(String contenido) { this.contenido =
16        contenido; }
17    public String getNombre() { return nombre; }
18    public String getContenido() { return contenido; }
19
20    @Override
21    public String toString() {
22        return "Archivo{" + "nombre=" + nombre + ", contenido=" +
23            contenido + '}';
24    }
25 }
```

2.1.2. Clase Impresora

Descripción

Define el hardware disponible en el sistema. Almacena el nombre y modelo del dispositivo.

Código Fuente

```
1 package impresora.singleton;
2
3 public class Impresora {
4
5     private String nombre;
```

```
6     private String modelo;
7     private ColaImpresion imprimir;
8
9     public Impresora(String nombre, String modelo) {
10         this.nombre = nombre;
11         this.modelo = modelo;
12     }
13
14     public void setNombre(String nombre) { this.nombre = nombre;
15     }
16     public void setModelo(String modelo) { this.modelo = modelo;
17     }
18     public String getNombre() { return nombre; }
19     public String getModelo() { return modelo; }
20 }
```

Patrón Singleton - Conexion

Descripción

La clase Conexion implementa el patrón **Singleton** para garantizar que el sistema mantenga un único punto de acceso global a la impresora activa[cite: 11]. Protege la creación mediante un constructor privado y expone la instancia a través del método estático getInstance().

Código Fuente

```
1 package impresora.singleton;
2
3 public class Conexion {
4
5     private static Conexion instance;
6     private Impresora impr;
7
8     //Constructor Privado
9     private Conexion(Impresora impr) {
10         System.out.println("Se conecta la impresora "+impr.
11             getNombre());
12         this.impr= impr;
13     }
14
15     public static Conexion getInstance(Impresora impr){
16         if (instance== null){
17             instance = new Conexion(impr);
18         }else{
19             instance.impr = impr;
20             System.out.println("Inicializando la impresora " +
21                 impr.getNombre());
22         }
23     }
24 }
```

Explicación

- **Variable instance:** Es estática y mantiene la referencia única en todo el ciclo de ejecución.
- **Control de acceso:** Si la instancia es nula, invoca al constructor privado. Si ya existe, actualiza la variable `impr` con la nueva impresora recibida e informa que se está inicializando.

Patrón Multiton - ColaImpresion

Descripción

La clase `ColaImpresion` implementa el patrón **Multiton** (o un gestor de instancias limitadas). Controla y limita la cantidad máxima de conexiones u objetos que pueden instanciarse. En este caso, restringe a un máximo de 3 archivos simultáneos.

Código Fuente

```
1 package impresora.singleton;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class ColaImpresion {
7
8     private static List<ColaImpresion> instances = new ArrayList
9         <>();
10    private static int numCola = 0;
11    private static final int maxArchivos = 3;
12    private static List<Archivo> archivos= new ArrayList<>();
13
14    private ColaImpresion(Archivo arch){
15        numCola+=1;
16        System.out.println("Adregando_archivo_a_la_posicion:_"+
17            numCola+"de_la cola de impresion\n");
18    }
19
20    public static ColaImpresion getInstance(Archivo archivo){
21        archivos.add(archivo);
22        if (instances.size() < maxArchivos){
23            ColaImpresion nueva = new ColaImpresion(archivo);
24            instances.add(nueva);
25            return nueva;
26        } else
27            System.out.println("Cola_llena, reutilizando_la_ultima_
28                instancia_");
29        return instances.get(maxArchivos-1);
30    }
31 }
```

Explicación

- **Límite:** Se define mediante la constante estática `maxArchivos = 3`.
- **Registro de Instancias:** Utiliza un `ArrayList` estático para almacenar las instancias creadas.
- **Reutilización:** Cuando el tamaño de la lista de instancias alcanza el límite, el método `getInstance` bloquea la creación de nuevos objetos y retorna el último objeto disponible usando `instances.get(maxArchivos-1)`.

Código de Cliente

Clase Main

Código Fuente

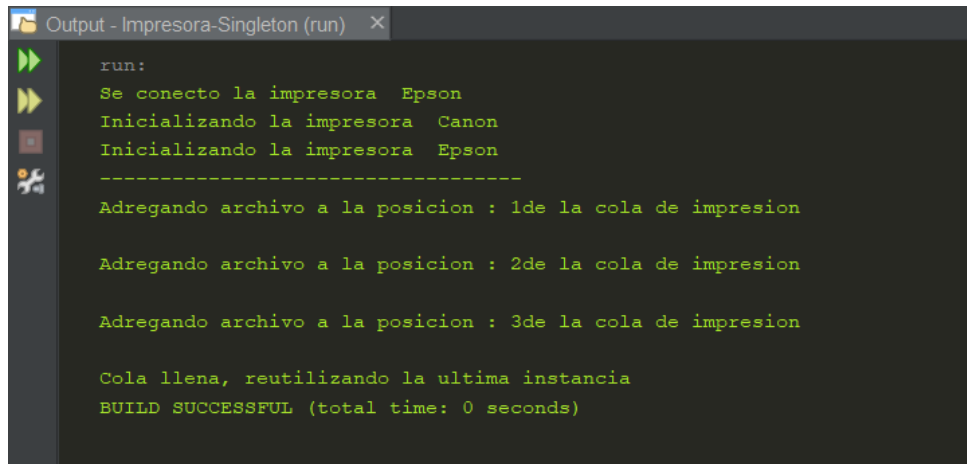
```
1 package impresora.singleton;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Impresora impr1 = new Impresora("Epson", "L3150");
8         Impresora impr2 = new Impresora("Canon", "G3110");
9         Conexion.getInstance(impr1);
10        Conexion.getInstance(impr2);
11        Conexion.getInstance(impr1);
12
13        System.out.println("-----");
14        ;
15
16        Archivo a1 = new Archivo("Doc1.pdf", "Texto1");
17        Archivo a2 = new Archivo("Doc2.pdf", "Texto2");
18        Archivo a3 = new Archivo("Doc3.pdf", "Texto3");
19        Archivo a4 = new Archivo("Doc4.pdf", "Texto4");
20
21        ColaImpresion.getInstance(a1);
22        ColaImpresion.getInstance(a2);
23        ColaImpresion.getInstance(a3);
24        ColaImpresion.getInstance(a4);
25    }
```

Flujo de Ejecución

1. Se crean los dispositivos Epson y Canon.
2. Se inicializa el **Singleton** enviando la impresora 1, luego se actualiza a la 2 y finalmente se retorna a la 1.
3. Se generan 4 objetos Archivo distintos.
4. Se envían los archivos al gestor **Multiton** (ColaImpresion). Los 3 primeros generan nuevas posiciones. Al enviar el cuarto (a4), el sistema informa que la cola está llena y reutiliza la última instancia disponible.

Pruebas y Ejecución

Prueba de Ejecución en Consola



```
run:
Se conecto la impresora Epson
Iniciando la impresora Canon
Iniciando la impresora Epson
-----
Adregando archivo a la posicion : 1de la cola de impresion

Adregando archivo a la posicion : 2de la cola de impresion

Adregando archivo a la posicion : 3de la cola de impresion

Cola llena, reutilizando la ultima instancia
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 2: Salida de la ejecución mostrando la inicialización del Singleton y el límite del Multiton.

Repositorio

Enlace GitHub

- <https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20creacionales/Impresora-Singleton>

Estructura de Paquetes

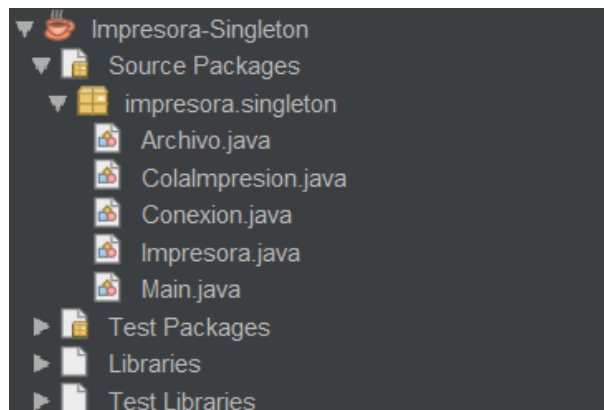


Figura 3: Árbol del proyecto en el IDE.